COMP 110

for loops + range()

Looping Through Sequences

- We commonly use loops to iterate over every element in a sequence.

for … in … loops

```
letters: list[str] = ["w", "x", "y", "z"]
```

Print every element of `letters`

while

for … in …

```
letters: list[str] = ["w", "x", "y", "z"]
```

```
while idx < len(letters):
```

iterates over

list[str]	
0	“w”
1	“x”
2	“y”
3	“z”

```
for elem in letters:
```

iterates over

```
idx: int = 0
while idx < len(letters):
    print(idx)
    idx += 1
```

Output:

0
1
2
3

```
for elem in letters:
    print(elem)
```

Output:

w
x
y
z

for … in … loops in Memory

```python
"""Practice of for Loops"""

my_list: list[str] = ["hello", "world"]
new_list: list[str] = []
for elem in my_list:
    new_list.append(elem)
print(new_list)
```

Writing for loops

- Let's implement a function named celebrate with one parameter:
  - **pet_names: list[str]** that stores the name of pets
- The return value of the function is None
- The function should use a **for … in** loop to print this string for every pet: “Good boy, <pet>!”
- Example:

```
>>> pets: list[str] = ["Louie", "Bo", "Bear"]
>>> celebrate(pets)
Good boy, Louie!
Good boy, Bo!
Good boy, Bear!
```

Why “while” loop over “for” loop?

Range

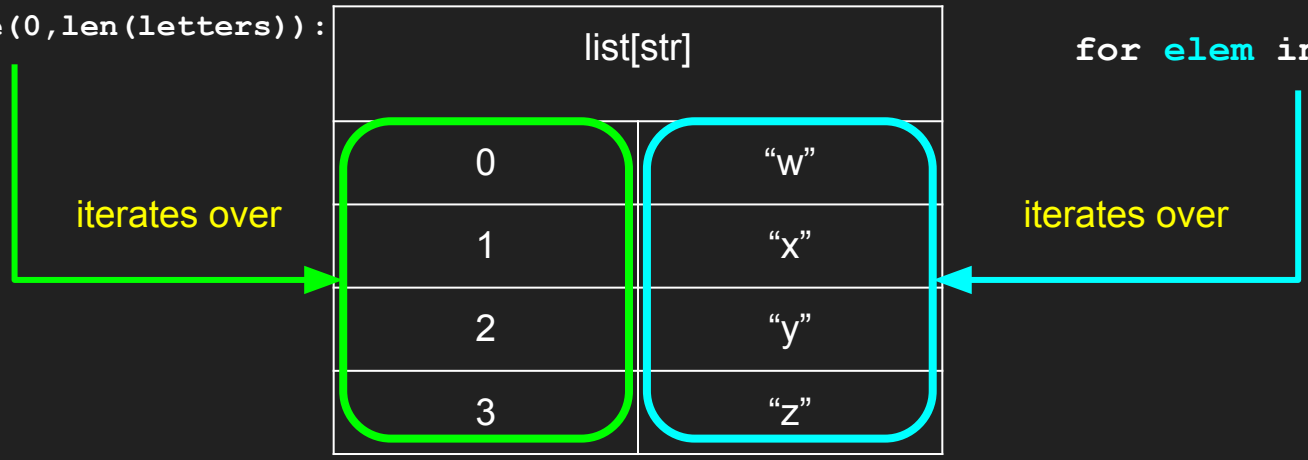
- A type of sequence you can loop over.
- Includes start point, does **<u>not</u>** include end point, and *steps* through every point in between
- Constructor: `range(start, end, [step = 1])`
- Examples:
  - `range(1, 5)` stops at numbers 1, 2, 3, 4
  - `range(1, 6, 2)` stops at numbers 1, 3, 5

range() in Memory

On the heap, but don’t worry about it. :-)

```
letters: list[str] = ["w", "x", "y", "z"]
```

```
for idx in range(0,len(letters)):
```




```
for elem in letters:
```

```
letters: list[str] = ["w", "x", "y", "z"]
```

```
for idx in range(0,len(letters)):
```

iterates over

list[str]	
0	“w”
1	“x”
2	“y”
3	“z”

```
for elem in letters:
```

iterates over

```
for idx in
range(0,len(letters)):
    print(idx)
```

Output:

0
1
2
3

```
for idx in
range(0,len(letters)):
    print(my_list[idx])
```

Output:

w
x
y
z

```
for elem in letters:
    print(elem)
```

Output:

w
x
y
z

Using range() in a for … in … loop.

```
names: list[str] = ["Alyssa", "Izzi", "Benjamin"]
```

Print every element’s index *and* value:

0: Alyssa

1: Izzi

2: Benjamin

Writing for…in range(...) loops

- Let's implement a function named celebrate_again with one parameter:
  - **pet_names: list[str]** that stores the name of pets
- The return value of the function is None
- The function should use a **for … in range(...)** loop to print this string for every pet: “Good boy, <pet>!”
- Example:

```
>>> pets: list[str] = ["Louie", "Bo", "Bear"]
>>> celebrate(pets)
Good boy, Louie!
Good boy, Bo!
Good boy, Bear!
```

“for” Loops + Dictionaries

“for” loops iterate over the *keys* by default

```
for key in ice_cream:
    print(key)
```

```
for key in ice_cream:
    print(ice_cream[key])
```

Flavor	Num Orders
“chocolate”	12
“vanilla”	8
“strawberry”	5

“for” Loops + Dictionaries

“for” loops iterate over the *keys* by default

```
for key in ice_cream:
    print(key)
```

```
for key in ice_cream:
    print(ice_cream[key])
```

Flavor	Num Orders
“chocolate”	12
“vanilla”	8
“strawberry”	5

Practice

- Let's implement a function named get_orders with one parameter:
  - **orders: dict[str, int]** that stores ice cream flavors as the keys and the number of orders as the values
- The return value of the function is None
- The function should print this string for every order in the dictionary: “<flavor> has <number> orders”
- Example:

```
>>> ice_cream: dict[str, int] = {"chocolate": 12, "vanilla": 8, "strawberry": 5}
>>> get_orders(ice_cream)
chocolate has 12 orders.
vanilla has 8 orders.
strawberry has 5 orders.
```

Diagram

```python
def group_names(names: list[str]) -> dict[str, int]:
    groups: dict[str, int] = {}
    first_letter: str
    for n in names:
        first_letter = n[0]
        if first_letter in groups:
            groups[first_letter] += 1
        else:
            groups[first_letter] = 1
    return groups

ppl: list[str] = ["Karen", "Emily", "Kris"]
result: dict[str, int] = group_names(names=ppl)
print(result)
result["I"] = 1
result.pop("E")
print(result)
```

Extra Practice

```python
def artist_counts(playlist: dict[str, str]) -> dict[str, int]:
    artists: dict[str, int] = dict()
    for track in playlist:
        art: str = playlist[track]
        if playlist[track] not in artists:
            artists[art] = 1
        else:
            artists[art] += 1
    return artists


songs: dict[str, str] = {
    "Showgirl": "Taylor",
    "Hello": "Erykah",
    "Fiat": "Butcher",
    "Woo": "Erykah",
}

print(artist_counts(songs))
```